
AI Agent for Operational Insights from a Large Operational Database

Planning & Implementation Documentation

Submitted by	Ajas Muhammed N
Role	AI / Backend Engineer Intern
Domain	Manufacturing & Logistics Operations
Duration	6 Weeks (6 × 1-Week Sprints)
Database	MySQL
Date	15 June 2026

Contents

List of Tables	ii
List of Figures	iii
1 Executive Summary	1
2 Project Objectives	2
3 High-Level System Architecture	3
3.1 Operational Database Schema	5
4 Architecture & Key Design Decisions	7
4.1 Natural-Language-to-SQL Pipeline	7
4.2 Conversational Memory	7
4.3 Schema-Aware Query Generation	7
4.4 Data Handling & Transparency	7
5 Implementation Plan	9
6 Recommended Technology Stack	10
7 Risks & Mitigations	11
8 Success Metrics & Acceptance Criteria	12

List of Tables

5.1 Implementation Plan 9

6.1 Recommended Technology Stack 10

7.1 Risk Register 11

List of Figures

3.1 AI Agent for Operational Insights — High-Level System Architecture . 4

3.2 Entity Relationship Diagram (ERD) of the Manufacturing Operations
Database 5

Chapter 1

Executive Summary

This document outlines the planning and implementation approach for an AI-powered conversational agent that surfaces operational insights directly from a manufacturing and logistics database. The agent allows operational stakeholders — floor managers, shift supervisors, procurement leads, and quality engineers — to query operational data in plain English and receive structured, natural-language answers, supported by automatically generated charts, without writing SQL or depending on a data engineering team.

A key design consideration is conversational continuity. The agent retains context within an active user session, allowing it to interpret natural follow-up questions without requiring the user to repeat information already supplied. This capability sits within the agent’s application layer and operates independently of the underlying database, so it adds no additional load to the database schema or performance.

The system is being developed over a structured six-week sprint plan, progressing from database design and synthetic data generation through to a fully functional natural-language interface with transparent SQL display and automatic chart generation.

Chapter 2

Project Objectives

- **Accessibility.** Allow any operational stakeholder to query the database in plain English, with no SQL expertise required.
- **Transparency.** Surface the generated SQL query to the user alongside every answer, supporting verification and trust.
- **Conversational Continuity.** Maintain context within a session so that follow-up questions can be asked naturally, without repeating earlier details.
- **Visualisation.** Automatically generate charts for numeric or trend-based query results, with no manual chart configuration.
- **Accuracy.** Correctly answer a defined benchmark set of operational business questions, against a minimum target threshold.
- **Deployability.** Support a simple, one-command local setup with no manual database configuration required after initial setup.

Chapter 3

High-Level System Architecture

The diagram below represents the architecture established during the design phase. It defines four principal layers — presentation, agent, LLM, and data — and the flow of information between the user's question and the final answer.

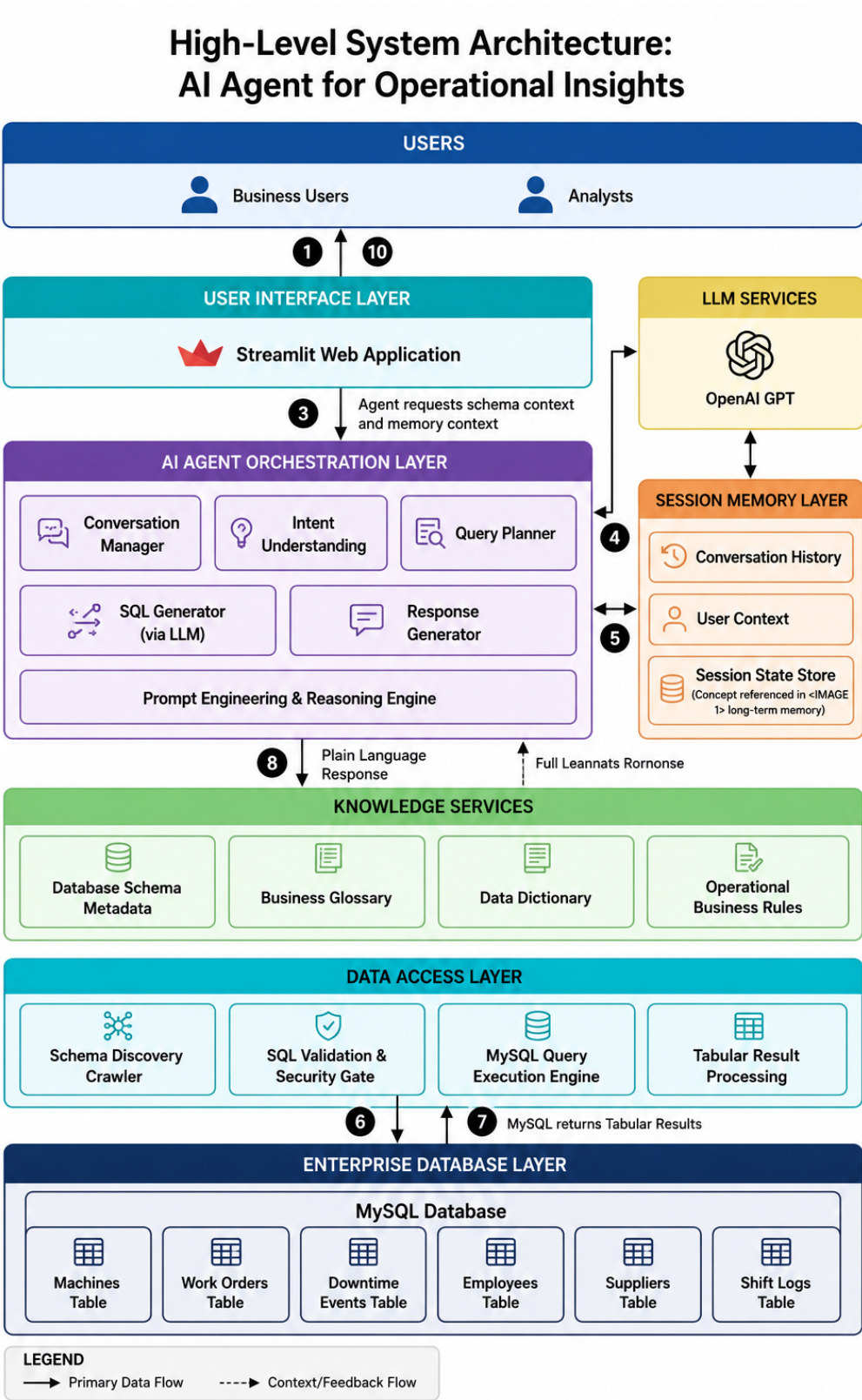


Figure 3.1. AI Agent for Operational Insights — High-Level System Architecture

Figure 3.1 presents the complete system architecture, illustrating the data flow across all four layers from user input through to response delivery.

User Question → [Agent: build prompt with schema context + session history] → [LLM API: generate SQL] → [Display SQL to user] → [Execute SQL on MySQL] → [Format result set] → [Generate chart where appropriate] → [Return plain English answer]

3.1 Operational Database Schema

The operational database serves as the core data layer of the system and has been implemented in MySQL. The schema consists of ten relational tables organized into four master tables (*machines*, *employees*, *suppliers*, *inventory*) and six transactional tables (*work_orders*, *maintenance_logs*, *downtime_events*, *purchase_orders*, *shift_logs*, *quality_checks*).

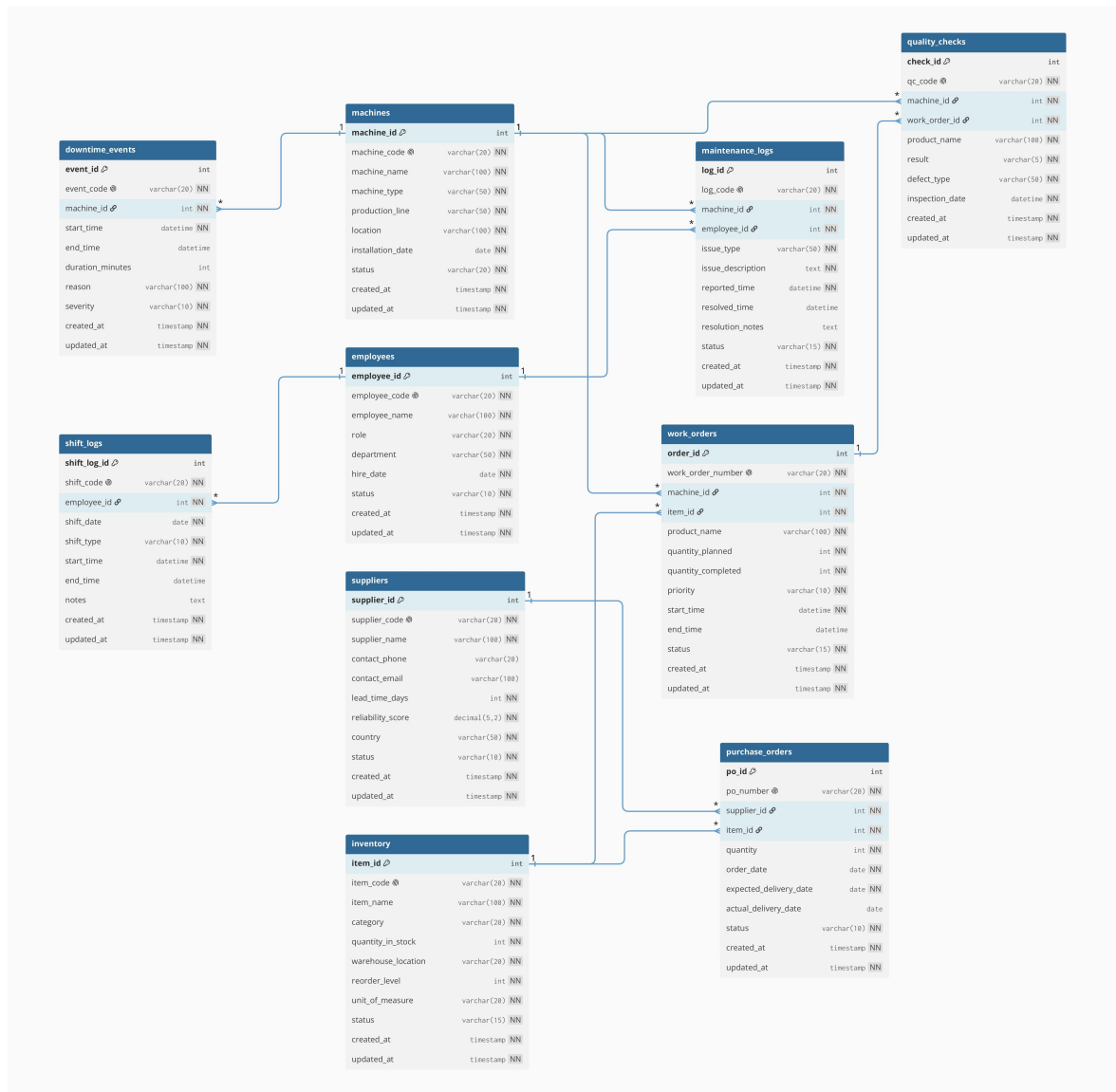


Figure 3.2. Entity Relationship Diagram (ERD) of the Manufacturing Operations Database

The schema was designed to support manufacturing operations analytics while maintaining referential integrity through foreign-key relationships and appropriate indexing strategies. The implemented schema contains ten tables, ten foreign-key relationships, business-key constraints, audit columns, and supporting indexes for operational analytics and future AI-driven query generation.

Chapter 4

Architecture & Key Design Decisions

4.1 Natural-Language-to-SQL Pipeline

The agent follows a five-stage processing pipeline for every user question: the question is received through the presentation layer; the agent constructs a prompt using relevant schema context and conversation history; the LLM returns a generated SQL statement; the query is displayed to the user for transparency; and the query is executed against the database, with the result formatted into a plain-English answer and, where appropriate, a chart. The specific LLM provider is being evaluated and finalised during the agent-development phase, with selection guided by SQL generation accuracy, reliability, and ease of integration.

4.2 Conversational Memory

The agent maintains conversational context for the duration of an active user session. This allows it to interpret follow-up questions — such as references to “that machine” or “the highest one” — without the user needing to repeat information already provided earlier in the conversation. Session context is held within the application layer and used to inform how each new question is interpreted, supporting a more natural, multi-turn interaction without adding complexity to the underlying database.

4.3 Schema-Aware Query Generation

Relevant schema information — table names, columns, and relationships — is provided to the LLM as part of the prompt, allowing it to generate accurate SQL against the operational schema. The generated query is always displayed to the user before and alongside the answer, giving full visibility into how each result was produced and supporting independent verification.

4.4 Data Handling & Transparency

All queries are executed as standard read operations against the operational database. Because every generated SQL statement is shown to the user, the system is built around

transparency rather than opacity — stakeholders and supervisors can audit any answer by reviewing the query that produced it. Error handling is included in the pipeline so that invalid or unanswerable questions return a clear, graceful message rather than a failure.

Chapter 5

Implementation Plan

The project is organised into two broad phases delivered across six one-week sprints.

Phase	Duration	Key Deliverables & Milestones
Phase 1 — Database Foundation	Weeks 1–3	Schema design and ERD, synthetic data generation with referential integrity, query validation and indexing across the 10-table operational schema.
Phase 2 — Agent & Interface	Weeks 4–6	LLM selection and core natural-language-to-SQL pipeline, conversational session memory, Streamlit interface with automatic chart generation, end-to-end testing and final report.

Table 5.1. Implementation Plan

Chapter 6

Recommended Technology Stack

Component	Technology	Rationale
Frontend / UI	Streamlit (Python)	Complete interactive web interface in pure Python — question input, SQL display, answers, charts, and session history — with no separate frontend framework.
Backend / Agent	Python 3.10+	Core agent logic: prompt construction, LLM calls, SQL execution, result formatting, and session memory.
Database	MySQL	Industry-standard relational database supporting joins, constraints, and indexing across a 10-table operational schema.
AI / LLM	To be finalised during agent development	Selection based on SQL generation accuracy, reliability, cost, and ease of integration.
Visualisation	Plotly	Interactive chart rendering integrated directly within Streamlit.
Data Generation	Faker (Python)	Generates realistic synthetic operational data while preserving referential integrity.
Version Control	GitHub	Source code, schema scripts, and documentation in a structured, auditable repository.
Dependency Management	requirements.txt	Enables one-command environment setup.

Table 6.1. Recommended Technology Stack

Chapter 7

Risks & Mitigations

Risk	Likelihood / Impact	Mitigation
SQL generation accuracy	Medium / High	Benchmark questions manually validated as ground-truth SQL before agent development; system prompt refined with examples to correct failure cases.
Synthetic data quality	Low / Medium	Business rules defined before generation; parent tables populated before child tables; referential integrity verified.
LLM API reliability	Medium / Medium	API reliability treated as a primary selection criterion; retry handling included in the pipeline.
Query performance on large datasets	Low / Medium	Indexes applied to high-frequency query columns; performance validated before the agent phase begins.

Table 7.1. Risk Register

Chapter 8

Success Metrics & Acceptance Criteria

- **Database.** 10+ tables with verified foreign-key relationships, populated with large-scale synthetic operational data.
- **Accuracy.** Agent correctly answers the defined benchmark set of business questions, against a minimum target threshold.
- **Visualisation.** A meaningful proportion of answers automatically generate a relevant chart.
- **Conversational Continuity.** Follow-up questions are correctly interpreted using session context.
- **Transparency.** Every answer is accompanied by the SQL query that produced it.
- **Deployability.** The application starts with a single setup command, with no manual database adjustment required afterward.